WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR SMART PARKING LOT ALLOCATION AND BILLING SYSTEM (ParkIQ)

Program	B.Tech in Computer Science and Engineering (AI & ML)
Course & Code	Database Management Systems (DBMS)
Section	AIML Rhinos
Batch	2025 - 2029
Course Faculty	Prof. Sahithi, School of Technology
Date of Submission	October 2026
Academic Year	2026 - 2027

Submitted by:

Roll Number	Student Name	Email ID
25WU0102340	G. Daiwik Reddy	[daiwik.g_2029@woxsen.edu.in]

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for giving me the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I express my heartfelt thanks to my course faculty, Prof. Sahithi, for continuous guidance, constructive suggestions and support throughout the design and development of this project, which was presented in three progressive reviews. The guidance helped me understand the practical use of core relational database concepts, including Entity-Relationship (ER) modelling, relational schema design, functional dependencies, normalization up to Third Normal Form (3NF), SQL constraint definition, stored procedures, triggers, transaction management and concurrency control.

This PBL activity has strengthened my understanding of database design and transactional integrity, and gave me hands-on experience in building a practical solution for the day-to-day operations of a smart parking lot.

UNDERTAKING

I hereby declare that the project entitled "DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR SMART PARKING LOT ALLOCATION AND BILLING SYSTEM", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of my course faculty, Prof. Sahithi.

I undertake that the project report, database design, SQL scripts, application source code, results and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date: October 14, 2026
Place: Hyderabad, Telangana
Signature:
Name: G. Daiwik Reddy

Program: B.Tech CSE (AI & ML) - AIML Rhinos
Roll Number: 25WU0102340
Academic Year: 2026-2027, School of Technology, Woxsen University

TABLE OF CONTENTS

- Abstract & Keywords
- List of Tables
- List of Figures
- List of Acronyms
- 1. Problem Statement, Solution Strategy and ER Diagram
  - 1.1 Problem Statement
  - 1.2 Objectives
  - 1.3 Scope and Assumptions
  - 1.4 Proposed Solution Strategy
  - 1.5 Functional Requirements
  - 1.6 Entity-Relationship Diagram (ERD)
  - 1.7 Entity and Relationship Summary
- 2. Logical Database Design
  - 2.1 Relational Schema
  - 2.2 Keys and Integrity Constraints
  - 2.3 Functional Dependencies
  - 2.4 Normalization up to Third Normal Form (3NF)
  - 2.5 Data Dictionary
  - 2.6 Implementation of Logical Design: DDL, Procedures and Triggers
- 3. Application Development Using Logical Database Design
  - 3.1 Application Overview
  - 3.2 Development Environment and Database Connectivity
  - 3.3 Application Modules and Database Mapping
  - 3.4 CRUD Operations
  - 3.5 Business Rules, Validation and Error Handling
  - 3.6 Reports and Database Operations
  - 3.7 Application Screens

    - 3.8 Testing and Results
- 4. Conclusion
    - 4.1 Limitations and Future Enhancements
- 5. References
- 6. Appendices
    - Appendix A - Database Script (Key Objects)
    - Appendix B - Application Source Code (Core Functions)
    - Appendix C - Demonstration Output and Test Data

LIST OF TABLES

Table	Title
Table 1	Scope and Assumptions
Table 2	Proposed Solution Strategy
Table 3	Functional Requirements
Table 4	Entity and Relationship Summary
Table 5	Relational Schema
Table 6	Keys and Integrity Constraints
Table 7	Normalization Status up to 3NF
Table 8	Comprehensive Data Dictionary
Table 9	Database Implementation Environment
Table 10	Application Development Environment
Table 11	Application Module-to-Database Mapping
Table 12	CRUD Operations
Table 13	Business Rules, Validation and Error Handling
Table 14	Reports and Database Operations
Table 15	Test Cases and Verification Outcomes

LIST OF FIGURES

Figure	Title
Figure 1	Entity-Relationship Diagram of the Proposed System
Figure 2	System Architecture of the Application
Figure 3	Registration, Entry and Slot Allocation Output
Figure 4	Reservation, Concurrency and Billing Output
Figure 5	Operational Reports Output
Figure 6	Parking Analytics Charts

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce-Codd Normal Form
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
EV	Electric Vehicle
FD	Functional Dependency

Acronym	Expansion
FK	Foreign Key
PK	Primary Key
RBAC	Role-Based Access Control
RDBMS	Relational Database Management System
SQL	Structured Query Language
UPI	Unified Payments Interface

ABSTRACT

Parking lots handle a continuous flow of vehicles, slots, tickets, reservations and payments. When these are managed by eye, on paper tickets and with hand-timed receipts, drivers circle the lot because free slots are not tracked live, two vehicles can be sent to the same slot, fees are calculated wrongly or disputed, and the owner has no stored data from which to study occupancy or revenue. The objective of this project is to design, normalize and implement a Relational Database Management System (RDBMS) named ParkIQ for smart parking lot allocation and billing.

The design evolved over three reviews: a four-table ER design with basic allocation and flat hourly billing (Review 1), an eight-table schema normalized to 3NF with a smart allocation procedure, tariff-based billing, triggers and test cases (Review 2), and a final thirteen-table system with concurrency-safe allocation, reservations, dynamic pricing, monthly passes, overstay penalties, role-based security, an audit log, views and reports (Review 3). Critical business rules, namely one open ticket per vehicle, one slot per vehicle at a time, no double allocation under concurrent entries, a valid exit time, a fee derived from the stored tariff and one payment per ticket, are enforced through declarative constraints, stored procedures and triggers. The MySQL 8 script defines the complete database, and a web application with an in-browser relational engine applies the same rules to demonstrate registration, entry, reservation, exit and payment, tariff management and reports. A test suite of eighteen cases, including negative integrity tests, was executed and all cases behaved as specified.

KEYWORDS

Relational Database, ER Diagram, Normalization, SQL, MySQL, Smart Parking, Slot Allocation, Billing, Triggers, Concurrency Control, Integrity Constraints

CHAPTER 1: PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Parking lots in campuses, offices, malls and residential complexes run many sessions in parallel: vehicles of different types arrive, slots are assigned, reservations are held in advance, vehicles leave, and a fee is collected at the exit. In many lots this information still lives on paper tickets, registers and hand-timed receipts. Nothing links an owner to a vehicle, a vehicle to its parking history, or a session to the slot it used and the money it earned.

This fragmentation causes several recurring problems. Drivers circle the lot because free slots are not tracked live. Without automated allocation, an attendant may send two vehicles to the same slot, or send a bike to a car bay while a bike bay is empty. Manual timing and paper receipts cause wrong or disputed charges, and past prices cannot be verified. Monthly pass holders and reservation holders are hard to recognize at the gate, and vehicles that overstay are not penalized. Without stored records, owners cannot study peak-hour occupancy, average stay or daily revenue. There is therefore a clear need for a centralized, normalized relational database that acts as one reliable source of truth for slots, vehicles, tickets and payments, enforces integrity rules, allocates slots safely and automates billing.

1.2 Objectives

The primary objectives of this project are:

1. Design a conceptual Entity-Relationship (ER) model covering the thirteen core entities of a smart parking lot, with precise cardinalities.
2. Derive the relational schema and normalize it to Third Normal Form (3NF) using functional dependencies, removing insertion, deletion and update anomalies.
3. Implement DDL constraints, stored procedures and triggers that enforce the key business rules: automatic allocation of the nearest free slot of the correct type, one open ticket per vehicle, no double allocation, valid entry and exit times, and one

payment per ticket.

4. Implement a billing engine that computes the fee from the stay duration, grace period, tariff rate, daily cap, peak-hour multiplier, membership discount and overstay penalty, with all rates held in a table.
5. Provide role-based access control and an audit log, and produce SQL views and reports for available slots, active sessions, daily revenue, peak-hour occupancy and average stay.
6. Build an application that supports the complete workflow from vehicle registration to payment with clear error handling.

1.3 Scope and Assumptions

The scope boundaries and technical assumptions of this implementation are summarized in Table 1.

Table 1. Scope and Assumptions

Item	Description
In Scope	User and vehicle registration; parking lot and slot master data with live status; slot reservation with automatic expiry; entry and exit tickets; concurrency-safe automatic slot allocation; tariff-based billing with grace period, daily cap and peak pricing; monthly passes; overstay penalties; payment records; staff roles; audit log; simulated slot sensor events; reports and views.
Out of Scope / Exclusions	Real sensors and number-plate recognition; QR-code tickets and a mobile app; live online payment gateway (UPI integration); SMS or email notifications; multi-lot cloud deployment.

Item	Description
Assumptions	One lot is modelled (Main Lot) with CAR, BIKE and EV slots; a vehicle occupies one slot at a time; a ticket is paid once, at exit; reservations are held for 15 minutes; the grace period is 15 minutes; peak hours are 08:00-10:59 and 17:00-19:59 by exit time with a 1.5x multiplier; a monthly pass gives 20% off; an overstay beyond 12 hours by a pass or reservation holder adds a penalty of Rs. 100; rates are sample values stored in the Tariff table.

1.4 Proposed Solution Strategy

The solution replaces paper tickets and manual timing with a single relational database and an application that works on it. Business rules are placed inside the database wherever possible so that they apply to every route into the data. Table 2 outlines the strategy.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Driver, Attendant and Admin (with a read-only Auditor for reports and the audit log).
Modules	Vehicle Registration; Slot Management and Allocation; Reservations; Entry and Exit Tickets; Billing and Payment; Security and Audit; Reports.
Proposed Workflow	Vehicle is registered -> on arrival, the free slot of the matching type with the lowest floor and shortest distance to the gate is locked and assigned (or the reserved slot is claimed) -> a ticket is issued and the slot becomes OCCUPIED -> at exit the exit time is recorded and the fee is computed from duration, tariff, peak multiplier and pass discount -> payment is stored and the slot becomes AVAILABLE again.

Solution Element	Project-Specific Description
Important Business Rules	A slot holds one vehicle at a time; one open ticket per vehicle; entry returns LOT FULL when no matching slot is free; two simultaneous entries never receive the same slot; exit time must be later than entry time; unclaimed reservations expire; a ticket is paid once; only admin changes tariffs and corrects payments.
Expected Reports / Outputs	Available slots by type; vehicles parked now; daily revenue; peak-hour occupancy; average stay by vehicle type; most-used slot types; audit trail.

1.5 Functional Requirements

The functional requirements of the system are listed and prioritized in Table 3.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	The system shall store user profiles and the vehicles (plate number and type) registered to each user.	High / Core Master Data
FR-02	The system shall maintain parking lots and slots with type, floor, distance to the entry and live status.	High / Core Master Data
FR-03	The system shall allocate the nearest free slot of the matching vehicle type automatically and reply LOT FULL when none exists.	High / Core Workflow
FR-04	The system shall prevent double allocation when several vehicles enter at the same time.	High / Critical Integrity Rule
FR-05	The system shall allow a slot to be reserved for a limited time and release it automatically if it is not claimed.	Medium / Workflow

Req. ID	Functional Requirement	Remarks / Priority
FR-06	The system shall issue one entry ticket per session, reject a second open ticket for the same vehicle and reject an exit time that is not later than the entry time.	High / Integrity Rule
FR-07	The system shall calculate the fee from the stay duration using the grace period, hourly rate and daily cap held in the Tariff table.	High / Financial Accuracy
FR-08	The system shall apply a peak-hour multiplier, a monthly-pass discount and an overstay penalty to the fee.	Medium / Dynamic Pricing
FR-09	The system shall record one payment per ticket and mark the slot AVAILABLE when the payment is saved.	High / Workflow
FR-10	The system shall restrict actions by role and record changes to payments and tariffs in an audit log.	High / Security
FR-11	The system shall provide views and reports for available slots, active sessions, daily revenue, peak-hour occupancy and average stay.	High / Business Analytics

1.6 Entity-Relationship Diagram (ERD)

The conceptual design is shown in Figure 1. The ER diagram models thirteen entities with their primary keys, foreign keys and cardinalities. A user owns many vehicles, a vehicle has many tickets, reservations and passes, and a parking lot has many slots. A slot is used by many tickets over time. The parking ticket is the hub of the workflow: it belongs to one vehicle and one slot, is optionally created from a reservation, is settled by exactly one payment and may carry penalties. Tariff supplies the rate used by each payment. Slot sensor events record every change of slot state. Staff and the audit log support security.

The ER design grew in stages. Review 1 modelled Vehicle, ParkingSlot, ParkingTicket and Payment. Review 2 added User, ParkingLot, Reservation and Tariff. Review 3 added SlotSensorEvent, Membership, Penalty, Staff and AuditLog.

Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 documents the entity catalog with attributes, cardinalities and business rules.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
USER	user_id (PK), full_name, phone, email	Owner master; phone is unique.
VEHICLE	vehicle_id (PK), user_id (FK), plate_no, vehicle_type; N:1 with USER	Registered vehicles; plate is unique; type is CAR, BIKE or EV.
PARKING_LOT	lot_id (PK), lot_name, location, total_slots	Lot master.
PARKING_SLOT	slot_id (PK), lot_id (FK), slot_no, slot_type, floor_no, dist_to_entry, status; N:1 with PARKING_LOT	Slots with live AVAILABLE / RESERVED / OCCUPIED status; slot number unique within a lot.
RESERVATION	reservation_id (PK), vehicle_id (FK), slot_id (FK), reserved_from, expires_at, status; N:1 with VEHICLE and SLOT	Advance hold; status HELD, CLAIMED or EXPIRED.
PARKING_TICKET	ticket_id (PK), vehicle_id (FK), slot_id (FK), reservation_id (FK, optional), entry_time, exit_time, status; N:1 with VEHICLE and SLOT	One record per parking session; CHECK exit_time > entry_time.
SLOT_SENSOR_EVENT	event_id (PK), slot_id (FK), event_type, event_time; N:1 with SLOT	Simulated sensor log of slot changes.
TARIFF	tariff_id (PK), vehicle_type, rate_per_hour, daily_cap, grace_min, peak_mult	One tariff row per vehicle type; changed without code changes.

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
PAYMENT	payment_id (PK), ticket_id (FK), tariff_id (FK), amount, pay_mode, paid_at; 1:1 with TICKET	Fee paid for a completed session; one payment per ticket.
MEMBERSHIP	membership_id (PK), vehicle_id (FK), valid_from, valid_to, discount_pct; N:1 with VEHICLE	Monthly pass that discounts every bill.
PENALTY	penalty_id (PK), ticket_id (FK), amount, reason; N:1 with TICKET	Overstay charge.
STAFF	staff_id (PK), username, role	Staff accounts with role admin, attendant or auditor.
AUDIT_LOG	log_id (PK), actor, action, table_name, row_id, old_value, new_value, logged_at	Trail of changes to payments and tariffs.

CHAPTER 2: LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was mapped to a relational schema of thirteen relations. Primary and foreign keys are defined for every relation to preserve referential integrity. Table 5 lists the relations grouped as in the final architecture.

Table 5. Relational Schema

S.No.	Table Name	Group	Purpose
1	User	Core	Stores owner profiles and contact details.
2	Vehicle	Core	Stores vehicles registered to users, with plate and type.
3	ParkingLot	Core	Stores parking lots and their capacity.
4	ParkingSlot	Core	Stores slots with type, floor, distance to entry and live status.

S.No.	Table Name	Group	Purpose
5	Reservation	Allocation	Stores advance holds on slots with expiry time.
6	ParkingTicket	Allocation	Stores one record per parking session.
7	SlotSensorEvent	Allocation	Stores simulated sensor events for slot changes.
8	Tariff	Billing	Stores rate, daily cap, grace period and peak multiplier per vehicle type.
9	Payment	Billing	Stores the fee paid for a completed session.
10	Membership	Billing	Stores monthly passes and discount percentage.
11	Penalty	Billing	Stores overstay penalties.
12	Staff	Security	Stores staff accounts and roles.
13	AuditLog	Security	Stores a log of changes to payments and tariffs.

2.2 Keys and Integrity Constraints

Declarative and procedural constraints guarantee consistency and prevent illegal data states. Table 6 maps the constraints to each relation.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
User	PK / UNIQUE / NOT NULL	user_id / phone / full_name	Unique owner identity; unique phone.
Vehicle	PK / FK / UNIQUE / ENUM	vehicle_id / user_id / plate_no / vehicle_type	Vehicle belongs to a valid user; unique plate; type in CAR, BIKE, EV.
ParkingLot	PK / NOT NULL	lot_id / lot_name, total_slots	Unique lot identity.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
ParkingSlot	PK / FK / UNIQUE / ENUM	slot_id / lot_id / (lot_id, slot_no) / slot_type, status	Slot belongs to a valid lot; unique slot number in a lot; status in AVAILABLE, RESERVED, OCCUPIED.
Reservation	PK / FK / ENUM	reservation_id / vehicle_id, slot_id / status	Hold refers to a valid vehicle and slot; status in HELD, CLAIMED, EXPIRED.
ParkingTicket	PK / FK / CHECK / trigger	ticket_id / vehicle_id, slot_id, reservation_id / exit_time / trg_ticket_before_insert	Exit time after entry time; no second open ticket for a vehicle.
SlotSensorEvent	PK / FK	event_id / slot_id	Event refers to a valid slot.
Tariff	PK / UNIQUE / NOT NULL	tariff_id / vehicle_type / rate_per_hour, daily_cap	One tariff per vehicle type.
Payment	PK / FK / UNIQUE / ENUM	payment_id / ticket_id, tariff_id / ticket_id / pay_mode	One payment per ticket; mode in CASH, UPI, CARD.
Membership	PK / FK	membership_id / vehicle_id	Pass belongs to a valid vehicle.
Penalty	PK / FK	penalty_id / ticket_id	Penalty belongs to a valid ticket.
Staff	PK / UNIQUE / ENUM	staff_id / username / role	Unique username; role in admin, attendant, auditor.
AuditLog	PK / JSON	log_id / old_value, new_value	Immutable record of who changed what.

2.3 Functional Dependencies

The non-trivial functional dependencies (FDs) identified for each relation are given below. They justify the normalization decisions in Section 2.4.

1. User: user_id -> {full_name, phone, email}; phone -> {user_id, full_name, email}.
2. Vehicle: vehicle_id -> {user_id, plate_no, vehicle_type}; plate_no -> {vehicle_id, user_id, vehicle_type}.
3. ParkingLot: lot_id -> {lot_name, location, total_slots}.
4. ParkingSlot: slot_id -> {lot_id, slot_no, slot_type, floor_no, dist_to_entry, status}; {lot_id, slot_no} -> {slot_id, slot_type, floor_no, dist_to_entry, status}.
5. Reservation: reservation_id -> {vehicle_id, slot_id, reserved_from, expires_at, status}.
6. ParkingTicket: ticket_id -> {vehicle_id, slot_id, reservation_id, entry_time, exit_time, status}.
7. SlotSensorEvent: event_id -> {slot_id, event_type, event_time}.
8. Tariff: tariff_id -> {vehicle_type, rate_per_hour, daily_cap, grace_min, peak_mult}; vehicle_type -> {tariff_id, rate_per_hour, daily_cap, grace_min, peak_mult}.
9. Payment: payment_id -> {ticket_id, tariff_id, amount, pay_mode, paid_at}; ticket_id -> {payment_id, tariff_id, amount, pay_mode, paid_at}.
10. Membership: membership_id -> {vehicle_id, valid_from, valid_to, discount_pct}.
11. Penalty: penalty_id -> {ticket_id, amount, reason}.
12. Staff: staff_id -> {username, role}; username -> {staff_id, role}.
13. AuditLog: log_id -> {actor, action, table_name, row_id, old_value, new_value, logged_at}.

2.4 Normalization up to Third Normal Form (3NF)

Every relation was checked against the standard normal forms. Review 2 started from one wide ticket table, Ticket(ticket_id, plate_no, owner_name, owner_phone, slot_no, slot_type, rate, entry_time, exit_time, amount), in which owner, slot and rate repeat on every row and cause update anomalies.

- **1NF:** all attributes hold atomic values and there are no repeating groups. Repeated sessions of a vehicle are stored as rows in ParkingTicket.
- **2NF:** the relations are in 1NF and no non-prime attribute depends on part of a candidate key. Slot details moved to ParkingSlot and vehicle details to Vehicle.

- **3NF:** the relations are in 2NF and no non-prime attribute is transitively dependent on a key. Owner contact moved to User and rates to Tariff; a ticket stores only vehicle_id and slot_id.
- **BCNF:** for every non-trivial FD X -> Y listed in Section 2.3, X is a superkey, so all thirteen relations also satisfy BCNF.

Table 7 summarizes the result.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
User	Yes	Yes	Yes	Yes	Candidate keys: user_id, phone.
Vehicle	Yes	Yes	Yes	Yes	Owner details kept in User; only user_id stored.
ParkingLot	Yes	Yes	Yes	Yes	*total_slots is derivable by counting slots; kept as a documented denormalization for quick capacity display.
ParkingSlot	Yes	Yes	Yes	Yes	Candidate keys: slot_id, (lot_id, slot_no).
Reservation	Yes	Yes	Yes	Yes	Vehicle and slot details not repeated.
ParkingTicket	Yes	Yes	Yes	Yes	Stores only vehicle_id, slot_id and times.
SlotSensorEvent	Yes	Yes	Yes	Yes	Append-only event log.
Tariff	Yes	Yes	Yes	Yes	Candidate keys: tariff_id, vehicle_type; rates depend only on type.
Payment	Yes	Yes	Yes	Yes	amount is a stored billing snapshot, so later tariff changes do not alter past bills.
Membership	Yes	Yes	Yes	Yes	Discount depends only on membership_id.
Penalty	Yes	Yes	Yes	Yes	Separate from Payment so the reason stays visible.
Staff	Yes	Yes	Yes	Yes	Candidate keys: staff_id, username.
AuditLog	Yes	Yes	Yes	Yes	Append-only trail.

2.5 Data Dictionary

Table 8 gives the data dictionary for all attributes implemented in the database.

Table 8. Comprehensive Data Dictionary

Table	Attribute	Datatype	Key / Constraint	Description
User	user_id	INT	PK, AUTO_INCREMENT	Unique user identifier.
User	full_name	VARCHAR(80)	NOT NULL	Owner full name.
User	phone	VARCHAR(15)	NOT NULL, UNIQUE	Primary contact number.
User	email	VARCHAR(100)	-	Contact email address.
ParkingLot	lot_id	INT	PK, AUTO_INCREMENT	Unique lot identifier.
ParkingLot	lot_name	VARCHAR(60)	NOT NULL	Name of the lot.
ParkingLot	location	VARCHAR(120)	-	Block or address of the lot.
ParkingLot	total_slots	INT	NOT NULL	Number of slots in the lot.
Vehicle	vehicle_id	INT	PK, AUTO_INCREMENT	Unique vehicle identifier.
Vehicle	user_id	INT	FK, NOT NULL	Owner of the vehicle.
Vehicle	plate_no	VARCHAR(12)	NOT NULL, UNIQUE	Registration plate number.
Vehicle	vehicle_type	ENUM	NOT NULL	CAR, BIKE or EV.
ParkingSlot	slot_id	INT	PK, AUTO_INCREMENT	Unique slot identifier.
ParkingSlot	lot_id	INT	FK, NOT NULL	Lot the slot belongs to.
ParkingSlot	slot_no	VARCHAR(8)	NOT NULL, UNIQUE with lot_id	Printed slot number, e.g. C1-1.
ParkingSlot	slot_type	ENUM	NOT NULL	CAR, BIKE or EV.
ParkingSlot	floor_no	TINYINT	NOT NULL	Floor of the slot.
ParkingSlot	dist_to_entry	SMALLINT	NOT NULL	Distance to the gate, used for ordering.

Table	Attribute	Datatype	Key / Constraint	Description
ParkingSlot	status	ENUM	NOT NULL, DEFAULT 'AVAILABLE'	AVAILABLE, RESERVED or OCCUPIED.
Reservation	reservation_id	INT	PK, AUTO_INCREMENT	Unique reservation identifier.
Reservation	vehicle_id	INT	FK, NOT NULL	Vehicle holding the slot.
Reservation	slot_id	INT	FK, NOT NULL	Slot being held.
Reservation	reserved_from	DATETIME	NOT NULL	Time the hold started.
Reservation	expires_at	DATETIME	NOT NULL	Time the hold lapses.
Reservation	status	ENUM	NOT NULL, DEFAULT 'HELD'	HELD, CLAIMED or EXPIRED.
ParkingTicket	ticket_id	INT	PK, AUTO_INCREMENT	Unique ticket identifier.
ParkingTicket	vehicle_id	INT	FK, NOT NULL	Parked vehicle.
ParkingTicket	slot_id	INT	FK, NOT NULL	Slot used.
ParkingTicket	reservation_id	INT	FK, NULL	Reservation claimed, if any.
ParkingTicket	entry_time	DATETIME	NOT NULL	Time the vehicle entered.
ParkingTicket	exit_time	DATETIME	CHECK > entry_time, NULL while parked	Time the vehicle left.
ParkingTicket	status	ENUM	NOT NULL, DEFAULT 'OPEN'	OPEN or CLOSED.
SlotSensorEvent	event_id	INT	PK, AUTO_INCREMENT	Unique event identifier.
SlotSensorEvent	slot_id	INT	FK, NOT NULL	Slot that changed.
SlotSensorEvent	event_type	VARCHAR(20)	NOT NULL	OCCUPIED, FREED, RESERVED or HOLD_EXPIRED.
SlotSensorEvent	event_time	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of the event.

Table	Attribute	Datatype	Key / Constraint	Description
Tariff	tariff_id	INT	PK, AUTO_INCREMENT	Unique tariff identifier.
Tariff	vehicle_type	ENUM	NOT NULL, UNIQUE	Vehicle type the rates apply to.
Tariff	rate_per_hour	DECIMAL(8,2)	NOT NULL	Charge per billable hour.
Tariff	daily_cap	DECIMAL(8,2)	NOT NULL	Maximum base fee for each 24 hours.
Tariff	grace_min	SMALLINT	NOT NULL, DEFAULT 15	Free minutes at the start of a stay.
Tariff	peak_mult	DECIMAL(3,2)	NOT NULL, DEFAULT 1.50	Peak-hour multiplier.
Payment	payment_id	INT	PK, AUTO_INCREMENT	Unique payment identifier.
Payment	ticket_id	INT	FK, NOT NULL, UNIQUE	Ticket being paid.
Payment	tariff_id	INT	FK, NOT NULL	Tariff used for the fee.
Payment	amount	DECIMAL(8,2)	NOT NULL	Amount paid, including any penalty.
Payment	pay_mode	ENUM	NOT NULL	CASH, UPI or CARD.
Payment	paid_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of payment.
Membership	membership_id	INT	PK, AUTO_INCREMENT	Unique pass identifier.
Membership	vehicle_id	INT	FK, NOT NULL	Vehicle holding the pass.
Membership	valid_from	DATE	NOT NULL	Pass start date.
Membership	valid_to	DATE	NOT NULL	Pass end date.
Membership	discount_pct	TINYINT	NOT NULL, DEFAULT 20	Percentage discount on each bill.
Penalty	penalty_id	INT	PK, AUTO_INCREMENT	Unique penalty identifier.

Table	Attribute	Datatype	Key / Constraint	Description
Penalty	ticket_id	INT	FK, NOT NULL	Ticket that was penalized.
Penalty	amount	DECIMAL(8,2)	NOT NULL	Penalty amount.
Penalty	reason	VARCHAR(80)	-	Reason for the penalty.
Staff	staff_id	INT	PK, AUTO_INCREMENT	Unique staff identifier.
Staff	username	VARCHAR(40)	NOT NULL, UNIQUE	Login name.
Staff	role	ENUM	NOT NULL	admin, attendant or auditor.
AuditLog	log_id	INT	PK, AUTO_INCREMENT	Unique log identifier.
AuditLog	actor	VARCHAR(40)	-	Who made the change.
AuditLog	action	VARCHAR(10)	-	INSERT or UPDATE.
AuditLog	table_name	VARCHAR(30)	-	Table that changed.
AuditLog	row_id	INT	-	Row that changed.
AuditLog	old_value, new_value	JSON	NULL allowed	Values before and after the change.
AuditLog	logged_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Time of the change.

2.6 Implementation of Logical Design: DDL, Procedures and Triggers

The database implementation details are summarized in Table 9.

Table 9. Database Implementation Environment

Item	Details
DBMS Engine	MySQL 8 (InnoDB storage engine with ACID transactions, row-level locking and SKIP LOCKED support).

Item	Details
Database Objects	13 tables, 1 function (fn_calc_fee), 4 stored procedures, 1 scheduled event, 5 triggers, 5 views, 3 secondary indexes, 3 roles.
Reference Data Loaded by the Script	1 lot, 10 slots (5 CAR, 3 BIKE, 2 EV), 3 tariffs (CAR, BIKE, EV), 3 staff accounts.
Demonstration Data (Application)	6 vehicles, 9 completed sessions across three days, 2 vehicles parked at the time of the demonstration.
Test Suite	18 test cases covering valid operations and negative integrity tests.

Representative DDL for the ticket table, the slot-allocation query and the triggers are shown below. The key objects are given in Appendix A, and the complete script is in the file parkiq_schema.sql.

```
CREATE TABLE ParkingTicket(
  ticket_id INT PRIMARY KEY AUTO_INCREMENT,
  vehicle_id INT NOT NULL, slot_id INT NOT NULL, reservation_id INT NULL,
  entry_time DATETIME NOT NULL, exit_time DATETIME NULL,
  status ENUM('OPEN','CLOSED') NOT NULL DEFAULT 'OPEN',
  CHECK(exit_time IS NULL OR exit_time > entry_time),
  FOREIGN KEY(vehicle_id) REFERENCES Vehicle(vehicle_id),
  FOREIGN KEY(slot_id) REFERENCES ParkingSlot(slot_id),
  FOREIGN KEY(reservation_id) REFERENCES Reservation(reservation_id));

-- Concurrency-safe choice of the nearest free slot
SELECT slot_id FROM ParkingSlot
 WHERE status = 'AVAILABLE' AND slot_type = 'CAR'
 ORDER BY floor_no, dist_to_entry
 LIMIT 1 FOR UPDATE SKIP LOCKED;

CREATE TRIGGER trg_ticket_before_insert BEFORE INSERT ON ParkingTicket FOR
EACH ROW
BEGIN
  IF EXISTS(SELECT 1 FROM ParkingTicket WHERE vehicle_id = NEW.vehicle_id AND
exit_time IS NULL) THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle already has an open
ticket';
  END IF;
```

```
END;

CREATE TRIGGER trg_payment_after_insert AFTER INSERT ON Payment FOR EACH ROW
BEGIN
  UPDATE ParkingSlot SET status = 'AVAILABLE'
   WHERE slot_id = (SELECT slot_id FROM ParkingTicket WHERE ticket_id =
NEW.ticket_id);
  INSERT INTO AuditLog(actor, action, table_name, row_id, new_value)
  VALUES (USER(), 'INSERT', 'Payment', NEW.payment_id, JSON_OBJECT('amount',
NEW.amount));
END;
```

CHAPTER 3: APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The front-end application is a web application that demonstrates how the normalized database is used in daily operations. It lets an attendant register vehicles, allocate slots on entry, reserve slots and close tickets at exit with automatic fee calculation; lets the admin manage tariffs and correct payments; and lets the owner or an auditor view reports and the database tables. The application contains an in-browser relational engine that holds the same thirteen tables and applies the same rules as the MySQL script, including the nearest-slot ordering, the skip-locked allocation, the open-ticket trigger, the exit-time check, the billing function and role permissions. Every rejected action is rolled back with a clear message, and a lot clock lets peak hours, grace periods and reservation expiry be tested without waiting.

3.2 Development Environment and Database Connectivity

Table 10. Application Development Environment

Item	Details
Programming Language	HTML5, CSS and JavaScript for the application; SQL (MySQL dialect) for the database.

Item	Details
DBMS & Storage	MySQL 8 for the database design and script; an in-browser relational engine with the same thirteen tables for the demonstration.
IDE / Tooling	Visual Studio Code; MySQL Workbench or draw.io for ER diagrams.
Database Connector / Driver	The demonstration application uses its own in-browser engine. Connecting the same screens to MySQL needs a driver such as Python mysql-connector (DB-API 2.0).
Operating Environment	Cross-platform; any modern web browser.
Security & Integrity Protocols	Role-based permissions (admin, attendant, auditor); transactions with commit and rollback; row locking with SKIP LOCKED; audit log for payments and tariffs; scheduled database dumps for backup.

3.3 Application Modules and Database Mapping

Figure 2 shows the architecture of the application, and Table 11 maps each module to the tables and operations it uses.

Figure 2. System Architecture of the Application

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Vehicle Registration	Register owners, vehicles and monthly passes.	User, Vehicle, Membership	INSERT with UNIQUE and FK validation.
Entry and Slot Allocation	Allocate the nearest free slot and issue a ticket.	Vehicle, ParkingSlot, ParkingTicket, Reservation, SlotSensorEvent	SELECT ... FOR UPDATE SKIP LOCKED; INSERT ticket; trigger sets OCCUPIED.

Module / Screen	Purpose	Tables Used	Main Operations
Reservations	Hold a slot for 15 minutes and expire it if unclaimed.	Reservation, ParkingSlot, SlotSensorEvent	INSERT reservation; UPDATE slot to RESERVED; expiry returns AVAILABLE.
Exit and Billing	Close the ticket, compute the fee and record payment.	ParkingTicket, Tariff, Membership, Penalty, Payment, ParkingSlot	UPDATE exit time; fn_calc_fee; INSERT payment; trigger sets AVAILABLE.
Tariff and Payment Admin	Change rates and correct payments (admin only).	Tariff, Payment, AuditLog	UPDATE with role check; audit rows written.
Reports	Operational reports for the owner.	All tables through views	SELECT with JOIN, GROUP BY and aggregates.
Database Viewer and Tests	Inspect all tables and run the test suite.	All tables	SELECT; scripted scenarios on a fresh database.

3.4 CRUD Operations

The application supports Create, Read and Update operations on all operational data. Records are not physically deleted during normal work: tickets, reservations and slots move through status values so that the parking history is preserved. Table 12 details these operations.

Table 12. CRUD Operations

Operation	Application Feature	Table(s) Affected	Evidence
Create	Registration, entry, reservation, payment, passes	User, Vehicle, ParkingTicket, Reservation, Payment, Membership, Penalty	INSERT INTO ParkingTicket (vehicle_id, slot_id, entry_time) VALUES (?, ?, ?)

Operation	Application Feature	Table(s) Affected	Evidence
Read / Search	Slot map, open sessions, fee quote, all reports	All tables; joins across ticket, vehicle and slot	SELECT plate_no, slot_no, entry_time FROM vw_active_sessions
Update	Slot status changes, ticket closure, reservation claim or expiry, tariff and payment correction	ParkingSlot, ParkingTicket, Reservation, Tariff, Payment	UPDATE Tariff SET rate_per_hour = ? WHERE vehicle_type = ?
Controlled Removal	Status change instead of deletion	ParkingSlot, ParkingTicket, Reservation	Status moves AVAILABLE -> OCCUPIED -> AVAILABLE; OPEN -> CLOSED; HELD -> CLAIMED or EXPIRED.

3.5 Business Rules, Validation and Error Handling

Table 13 shows how key business rules are enforced by layers of database constraints, triggers, procedures and application logic. Every write runs as one unit: it is saved on success, and on failure it is rolled back and the reason is shown to the user.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Unique plate number	UNIQUE on Vehicle.plate_no	Plate already registered
One open ticket per vehicle	BEFORE INSERT trigger trg_ticket_before_insert	Vehicle already has an open ticket
Nearest free slot of the right type	ORDER BY floor_no, dist_to_entry with status and slot_type filters	Ticket issued for slot C1-1 (floor 1)
No double allocation	Transaction with FOR UPDATE SKIP LOCKED	The second of two simultaneous entries gets LOT FULL
Lot full	No row returned by the allocation query; transaction rolled back	LOT FULL: no free CAR slot

Rule / Validation	How Implemented	Evidence / Message
Reservation expiry	sp_expire_reservations and a one-minute event	Hold EXPIRED; slot AVAILABLE
Valid times	CHECK (exit_time IS NULL OR exit_time > entry_time)	Rejected by CHECK (exit_time > entry_time)
One payment per ticket	UNIQUE on Payment.ticket_id	Second payment for a ticket is rejected
Slot freed on payment	AFTER INSERT trigger trg_payment_after_insert	Slot status becomes AVAILABLE
Role permissions	GRANT on roles pk_admin, pk_attendant, pk_auditor	Denied: role "attendant" is not allowed to tariff
Audit trail	AFTER UPDATE triggers on Payment and Tariff	AuditLog row with old and new values
Referential integrity	FOREIGN KEY constraints on all child tables	Row cannot be added for an unknown vehicle or slot

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Available Slots	View vw_available_slots: COUNT with GROUP BY on slot_type for AVAILABLE slots.	Free slots per lot and type.
Vehicles Parked Now	View vw_active_sessions: JOIN of ParkingTicket, Vehicle and ParkingSlot WHERE exit_time IS NULL.	Plate, slot and entry time.
Daily Revenue	View vw_daily_revenue: GROUP BY DATE(paid_at) with SUM(amount).	Revenue per day.
Peak-Hour Occupancy	View vw_peak_occupancy: GROUP BY HOUR(entry_time) with COUNT.	Entries per hour of day.

Report / Operation	SQL / Database Concept Used	Application Output
Average Stay	View vw_avg_stay: JOIN with AVG of TIMESTAMPDIFF(MINUTE, entry_time, exit_time) per vehicle type.	Average minutes per vehicle type.
Most-Used Slot Types	JOIN of ParkingTicket and ParkingSlot with GROUP BY slot_type.	Sessions per slot type.
Fee Calculation	fn_calc_fee: CEILING, LEAST, grace period, peak multiplier and membership subquery.	Fee stored in Payment.

3.7 Application Screens

Figures 3 to 6 show the output of the application during the scripted demonstration. The scenario follows one customer, Ravi Kumar, who drives a car (TS09AB1234) into the lot on 12 October 2026 at 12:00. Steps 1 to 5 run on a fresh database, and the reports in Figure 5 use the sample dataset of nine completed sessions. Lines marked [OK] succeeded and lines marked [BLOCKED] were rejected by the database rules.

```
--- Step 1-3: register, enter, allocate ---
[OK]      Register vehicle TS09AB1234 (CAR, Ravi Kumar)
[BLOCKED] Register vehicle TS09AB1234 again -> Plate already registered
[OK]      Register vehicle AP28EF9012 (BIKE, Imran Shaikh)
[OK]      Entry TS09AB1234 at 12:00 -> ticket 1, slot C1-1, floor 1
[BLOCKED] Entry TS09AB1234 again -> Vehicle already has an open ticket
[OK]      Entry AP28EF9012 at 12:00 -> ticket 2, slot B1-1
```

Figure 3. Registration, Entry and Slot Allocation Output

```
--- Step 4: reservation and concurrency (lot with one CAR slot) ---
[OK]      Reserve slot for AAAA1 -> slot C1-1 held for 15 minutes
[OK]      Entry AAAA1 -> claims held slot C1-1 (reservation CLAIMED)
[OK]      Two entries at once -> A gets C1-1, B gets LOT FULL (1 ticket
issued)
[OK]      Hold not claimed for 16 minutes -> reservation EXPIRED, next
vehicle gets C1-1

--- Step 5: exit and billing ---
[OK]      Exit TS09AB1234 at 14:40 (stay 2h 40m) -> 3 billable h x 30 = 90,
slot C1-1 AVAILABLE
[OK]      Exit of a pass holder, same stay -> 90 - 20% = 72
[OK]      Exit at 09:30 after a 07:00 entry -> 90 x 1.5 peak = 135
[BLOCKED] Exit one minute before entry -> Rejected by CHECK (exit_time >
entry_time)
```

Figure 4. Reservation, Concurrency and Billing Output

```
== Available slots ==          == Vehicles parked now ==
slot_type  free_slots           plate_no     slot_no  entry
CAR        5                    AP28EF9012   B1-1     11:30
BIKE       2                    TS08GH3456   E1-1     12:00
EV         1

== Daily revenue ==             == Average stay (minutes) ==
day          revenue            vehicle_type  avg_minutes  sessions
2026-10-10   213                CAR           148          6
2026-10-11   269                BIKE          60           2
2026-10-12   231                EV            130          1

== Peak-hour entries (hour: entries) ==
08:1  09:2  10:1  11:1  12:2  13:1  17:2  18:1

== Most-used slot types ==      Total revenue: 713
CAR 6   BIKE 3   EV 2
```

Figure 5. Operational Reports Output

Figure 6. Parking Analytics Charts

3.8 Testing and Results

Verification covered both valid workflows and negative tests that try to break the integrity rules. Each test builds a fresh database, runs the scenario and records the message or value returned. Table 15 lists the test cases with the actual results.

Table 15. Test Cases and Verification Outcomes

ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Vehicle registration	Register TS09AB1234 (CAR)	Row inserted	Registered TS09AB1234, vehicle id 1	PASSED
TC-02	Unique plate (negative)	Register TS09AB1234 again	UNIQUE rule rejects the row	Plate already registered	PASSED
TC-03	Nearest slot allocation	Enter CARs on an empty lot	Lowest floor and shortest distance first; slot becomes OCCUPIED	C1-1 (floor 1, distance 8, OCCUPIED); later cars C1-2, C1-3, then C2-1 on floor 2	PASSED
TC-04	Slot type matching	Enter a BIKE	BIKE slot assigned	B1-1 (BIKE)	PASSED
TC-05	Second open ticket (negative)	Enter the same vehicle twice	Trigger blocks the second ticket	Vehicle already has an open ticket; tickets = 1	PASSED
TC-06	Lot full (negative)	Sixth CAR enters a lot with five CAR slots	LOT FULL; no ticket created	LOT FULL: no free CAR slot; tickets = 5	PASSED
TC-07	Concurrent entries	Two entries, one free CAR slot	One ticket issued; the other gets LOT FULL	A got C1-1, B got LOT FULL, tickets = 1	PASSED
TC-08	Claim reservation	Reserve a slot, then enter	Held slot is claimed	Held C1-1, entered C1-1, reservation CLAIMED	PASSED
TC-09	Reservation expiry	Hold a slot, wait 16 minutes, another vehicle enters	Hold expires; slot returns to AVAILABLE	RESERVED before; EXPIRED after 16 minutes; other vehicle got C1-1	PASSED
TC-10	Grace period	CAR stays 10 minutes	Fee is Rs. 0	Fee 0; slot AVAILABLE	PASSED
TC-11	Standard fee	CAR parks 12:00 to 14:40	3 billable hours x 30 = 90	Fee 90; slot AVAILABLE	PASSED
TC-12	Daily cap	CAR parks 11:00 to 21:00	Fee limited to the daily cap	Fee 200 (10 billable hours would be 300)	PASSED

ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-13	Peak multiplier	CAR parks 07:00 to 09:30	90 x 1.5 = 135	Fee 135	PASSED
TC-14	Monthly pass discount	Pass holder parks 12:00 to 14:40	90 less 20% = 72	Fee 72 (discount 18)	PASSED
TC-15	Overstay penalty	Pass holder parks 06:00 to 19:30	(200 x 1.5 less 20%) + 100 = 340	Fee 340 (penalty 100)	PASSED
TC-16	Exit time order (negative)	Exit one minute before entry	CHECK rejects the update	Rejected by CHECK (exit_time > entry_time); ticket still open	PASSED
TC-17	Role permission (negative)	Attendant changes the CAR rate	Denied by role permissions	Denied: role "attendant" is not allowed to tariff; rate stays 30	PASSED
TC-18	Audit trail	Admin changes a payment from 60 to 10	Change recorded in AuditLog	UPDATE row with old {"amount":60} and new {"amount":10}	PASSED

All eighteen test cases behaved as specified. The negative tests show that duplicate plates, second open tickets, double allocation, invalid exit times and unauthorized changes are stopped by the rules themselves, and that billing produces the expected figures, for example 3 billable hours x Rs. 30 = Rs. 90 for the worked example and Rs. 135 when the exit falls in a peak hour.

CHAPTER 4: CONCLUSION

This Project-Based Learning activity completed the design, normalization, implementation and application development of ParkIQ, a database management system for smart parking lot allocation and billing. Thirteen entities were derived from the requirements across three reviews, modelled in an ER diagram and normalized to Third Normal Form. The key business rules, namely allocation of the nearest free slot of the right type, one open ticket per vehicle, no double allocation under concurrent entries, valid exit times, tariff-driven billing with grace period, daily cap, peak pricing, pass discount and overstay penalty, role-based access and an audit trail, are placed inside the database design through constraints, procedures, triggers and roles. A web application with an in-browser engine

delivers the full workflow from registration to payment, with reservations, tariff management and reports. The test suite confirmed the expected behaviour in all eighteen cases.

4.1 Limitations and Future Enhancements

The current implementation has the following limitations, which also point to future work:

1. In-browser demonstration engine: the application does not yet talk to a MySQL server. Connecting the same screens to MySQL needs a driver such as Python mysql-connector and parameterized queries.
2. Single lot and simulated sensors: real sensors, number-plate recognition and multi-lot cloud deployment are natural extensions.
3. Fixed peak window and penalty rule: the peak hours and the 12-hour overstay limit are written into the billing logic, and could move into configuration tables like the rates.
4. Role selector instead of login: staff authentication with passwords or single sign-on should replace the role selector, using the MySQL roles already defined.
5. No mobile app, QR-code tickets, online payment or notifications: a mobile app with QR tickets, UPI payment and SMS or email reminders can be integrated.

---

5. REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[3] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.

[4] J. D. Ullman and J. Widom, A First Course in Database Systems, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

[5] Oracle Corporation, "MySQL 8.0 Reference Manual," MySQL. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] Python Software Foundation, "Python Database API Specification v2.0 (PEP 249)," Python.org. [Online]. Available: https://peps.python.org/pep-0249/

[7] ISO/IEC, "Information technology - Database languages - SQL," ISO/IEC 9075, Geneva, Switzerland.

---

6. APPENDICES

APPENDIX A - DATABASE SCRIPT (KEY OBJECTS)

The complete DDL, procedures, triggers, views, roles and seed data are in the file parkiq_schema.sql. The billing function and the allocation procedure are reproduced below.

```
CREATE FUNCTION fn_calc_fee(p_ticket INT, p_exit DATETIME) RETURNS
DECIMAL(8,2) READS SQL DATA
BEGIN
  DECLARE v_veh INT; DECLARE v_type VARCHAR(5); DECLARE v_entry DATETIME;
  DECLARE v_rate, v_cap, v_peak, v_fee DECIMAL(8,2); DECLARE v_grace, v_mins,
v_bill, v_disc INT;
  SELECT t.vehicle_id, v.vehicle_type, t.entry_time INTO v_veh, v_type,
v_entry
    FROM ParkingTicket t JOIN Vehicle v USING(vehicle_id) WHERE t.ticket_id =
p_ticket;
  SELECT rate_per_hour, daily_cap, grace_min, peak_mult INTO v_rate, v_cap,
v_grace, v_peak
    FROM Tariff WHERE vehicle_type = v_type;
  SET v_mins = TIMESTAMPDIFF(MINUTE, v_entry, p_exit);
  SET v_bill = CEILING(GREATEST(0, v_mins - v_grace) / 60);
  SET v_fee  = LEAST(v_bill * v_rate, v_cap * CEILING(GREATEST(v_mins,1) /
1440));
  IF v_bill > 0 AND (HOUR(p_exit) BETWEEN 8 AND 10 OR HOUR(p_exit) BETWEEN 17
AND 19) THEN
    SET v_fee = v_fee * v_peak;
  END IF;
  SET v_disc = IFNULL((SELECT MAX(discount_pct) FROM Membership
                       WHERE vehicle_id = v_veh AND valid_to >=
DATE(p_exit)), 0);
  RETURN ROUND(v_fee * (1 - v_disc / 100));
END;

CREATE PROCEDURE sp_allocate_slot(IN p_plate VARCHAR(12), OUT p_msg
VARCHAR(60))
BEGIN
```

```
  DECLARE v_veh INT; DECLARE v_type VARCHAR(5); DECLARE v_slot INT; DECLARE
v_res INT;
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; SET p_msg = 'ERROR:
entry rejected'; END;
  CALL sp_expire_reservations();
  SELECT vehicle_id, vehicle_type INTO v_veh, v_type FROM Vehicle WHERE
plate_no = p_plate;
  START TRANSACTION;
  SELECT reservation_id, slot_id INTO v_res, v_slot FROM Reservation
   WHERE vehicle_id = v_veh AND status = 'HELD' LIMIT 1;
  IF v_slot IS NULL THEN
    SELECT slot_id INTO v_slot FROM ParkingSlot WHERE status = 'AVAILABLE'
AND slot_type = v_type
      ORDER BY floor_no, dist_to_entry LIMIT 1 FOR UPDATE SKIP LOCKED;
  ELSE UPDATE Reservation SET status = 'CLAIMED' WHERE reservation_id =
v_res; END IF;
  IF v_slot IS NULL THEN ROLLBACK; SET p_msg = 'LOT FULL';
  ELSE
    INSERT INTO ParkingTicket(vehicle_id, slot_id, reservation_id,
entry_time) VALUES (v_veh, v_slot, v_res, NOW());
    COMMIT; SET p_msg = CONCAT('SLOT ', v_slot);
  END IF;
END;
```

APPENDIX B - APPLICATION SOURCE CODE (CORE FUNCTIONS)

The complete application is archived in the file parkiq.html. The core functions of the in-browser engine are shown below.

```
// SELECT ... FOR UPDATE SKIP LOCKED: skip rows locked by another entry
db.begin = ty => {
  const s = T.ParkingSlot
    .filter(s => s.status === 'AVAILABLE' && s.slot_type === ty &&
!db.locks.has(s.slot_id))
    .sort((a, b) => a.floor_no - b.floor_no || a.dist_to_entry -
b.dist_to_entry)[0];
  if (s) db.locks.add(s.slot_id);
  return s || null;
};
```

```
db.enter = plate => {
  need('enter'); expire(); const v = veh(plate);
  if (openT(v)) throw Error('Vehicle already has an open ticket');
  const res = T.Reservation.find(r => r.vehicle_id === v.vehicle_id &&
r.status === 'HELD');
  let s;
  if (res) { s = slot(res.slot_id); res.status = 'CLAIMED'; }
  else { s = db.begin(v.vehicle_type); if (!s) throw Error('LOT FULL: no free
' + v.vehicle_type + ' slot'); }
  return { t: db.commit(s, v, res), s };
};

db.quote = (t, at) => {
  const mins = (at - t.entry_time) / 60000, bill = Math.ceil(Math.max(0, mins
- tf.grace_min) / 60);
  const base = Math.min(bill * tf.rate_per_hour, tf.daily_cap *
Math.ceil(Math.max(mins, 1) / 1440));
  const h = new Date(at).getHours(), peak = bill > 0 && ((h >= 8 && h < 11)
|| (h >= 17 && h < 20));
  const sub = base * (peak ? tf.peak_mult : 1), disc = m ? sub *
m.discount_pct / 100 : 0;
  const pen = (m || t.reservation_id) && mins > 720 ? 100 : 0;
  return { total: Math.round(sub - disc) + pen };
};
```

APPENDIX C - DEMONSTRATION OUTPUT AND TEST DATA

The output of the scripted demonstration is reproduced in Figures 3 to 5. The raw results of the test suite, one line per case, are listed below.

```
TC-03  C1-1 floor 1 dist 8 status OCCUPIED; later cars C1-2, C1-3, C2-1
TC-06  LOT FULL: no free CAR slot | tickets=5
TC-07  A=C1-1, B=LOT FULL, tickets=1
TC-09  before RESERVED; after 16 min reservation EXPIRED; B got C1-1
TC-10  stay 10m, billable 0, base 0, peak x1, disc 0, pen 0, total 0
TC-11  stay 160m, billable 3, base 90, peak x1, disc 0, pen 0, total 90
TC-12  stay 600m, billable 10, base 200, peak x1, disc 0, pen 0, total 200
TC-13  stay 150m, billable 3, base 90, peak x1.5, disc 0, pen 0, total 135
TC-14  stay 160m, billable 3, base 90, peak x1, disc 18, pen 0, total 72
TC-15  stay 810m, billable 14, base 200, peak x1.5, disc 60, pen 100, total
340
TC-16  Rejected by CHECK (exit_time > entry_time) | open tickets = 1
TC-17  Denied: role "attendant" is not allowed to tariff | rate = 30
TC-18  AuditLog old {"amount":60} new {"amount":10}
```

Sample reference data: 1 lot (Main Lot); 10 slots (C1-1 to C1-3 and C2-1 to C2-2 for cars, B1-1 to B1-3 for bikes, E1-1 to E1-2 for EVs); tariffs CAR Rs. 30 per hour with cap 200, BIKE Rs. 15 with cap 100, EV Rs. 40 with cap 250; grace period 15 minutes; peak multiplier 1.5.